

Low-Level Design (LLD) Documentation

Functional Data Processing Pipeline with Apache Spark

Functional Data Processing Pipeline with Apache Spark

****Project:**** Functional Data Processing Pipeline

****Technology Stack:**** Scala 2.12.19, Apache Spark 3.5.0, sbt 1.9.7

****Java Version:**** OpenJDK 11

****Date:**** September 2025

Table of Contents

1. [System Overview](#system-overview)
2. [Architecture Design](#architecture-design)
3. [Data Models](#data-models)
4. [Component Design](#component-design)
5. [Functional Programming Implementation](#functional-programming-implementation)
6. [Data Flow](#data-flow)
7. [Configuration](#configuration)
8. [Testing Strategy](#testing-strategy)
9. [Deployment](#deployment)

System Overview

Purpose

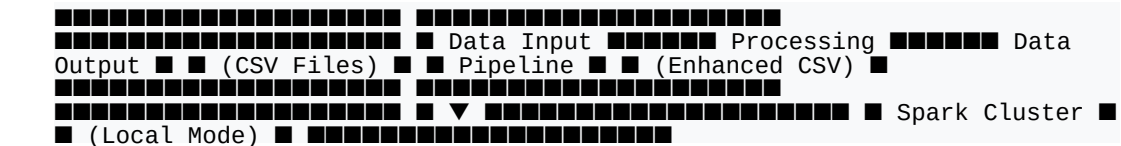
The Functional Data Processing Pipeline is a Scala-based application that processes movie data using Apache Spark, demonstrating advanced functional programming principles including immutability, pure functions, higher-order functions, currying, and combinators.

Key Features

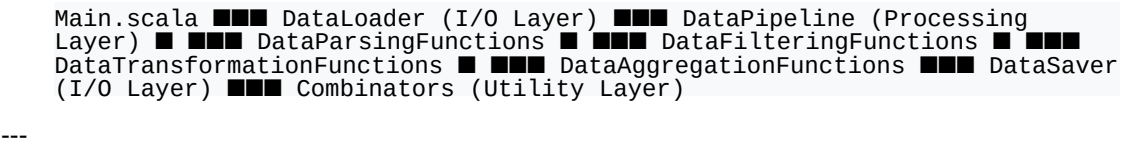
- **Data Ingestion**: CSV file parsing and validation
 - **Data Processing**: Functional transformations and enrichments
 - **Data Output**: Structured CSV output with enhanced metadata
 - **Quality Filtering**: High-quality movie selection criteria
 - **Analytics**: Genre statistics and decade-based analysis
-

Architecture Design

High-Level Architecture



Component Architecture



Data Models

Core Data Structures

1. DataRecord (Sealed Trait)

```
sealed trait DataRecord
```

- **Purpose**: Type-safe hierarchy for all data records
- **Benefits**: Enables pattern matching and type safety

2. MovieRecord (Input Model)

```
case class MovieRecord( id: String, title: String, year: Int, genre: String, rating: Double, votes: Long ) extends DataRecord
```

- **Purpose**: Represents raw movie data from CSV
- **Immutability**: All fields are immutable
- **Validation**: Parsed and validated from CSV input

3. ProcessedMovieRecord (Output Model)

```
case class ProcessedMovieRecord( movie: MovieRecord, decade: String, ratingCategory: String, popularityScore: Double ) extends DataRecord
```

- **Purpose**: Enriched movie data with additional metadata
- **Enrichment Fields**:
 - `decade`: Calculated from year (e.g., "1990s")
 - `ratingCategory`: Categorized rating ("Excellent", "Good", "Average")
 - `popularityScore`: Computed from rating and votes

4. Supporting Models

```
case class GenreStatistics( genre: String, movieCount: Int, averageRating: Double, topMovie: String ) case class ProcessingResult( totalMovies: Int, processedMovies: Int, genreStats: List[GenreStatistics] ) case class PipelineConfig( minRating: Double, minVotes: Long, topMoviesCount: Int )
```

Component Design

1. Main.scala (Entry Point)

Responsibilities:

- Initialize Spark session
- Orchestrate data processing pipeline
- Handle application lifecycle

Key Methods:

```
def main(args: Array[String]): Unit
```

Design Patterns:

- **Dependency Injection**: Spark session passed to components
- **Resource Management**: Try-finally for proper cleanup

2. DataLoader (I/O Layer)

Responsibilities:

- Load raw data from external sources
- Parse CSV files into Spark Datasets
- Handle file system operations

Key Methods:

```
def loadData(spark: SparkSession, inputPath: String): Dataset[String]
```

Design Principles:

- **Separation of Concerns**: I/O separated from business logic
- **Error Handling**: Graceful handling of file system errors

3. DataPipeline (Processing Layer)

Responsibilities:

- Orchestrate data processing workflow
- Apply functional transformations
- Coordinate between different processing functions

Key Methods:

```
def processData(rawData: Dataset[String]): Dataset[ProcessedMovieRecord]
def computeGenreStatistics(movies: Dataset[ProcessedMovieRecord]):
Dataset[GenreStatistics] def findTopMoviesByDecade(movies:
Dataset[ProcessedMovieRecord]): Dataset[(String,
List[ProcessedMovieRecord])]
```

Design Patterns:

- **Pipeline Pattern**: Sequential data processing stages
- **Functional Composition**: Chaining pure functions

4. DataSaver (I/O Layer)

Responsibilities:

- Save processed data to external destinations
- Handle data serialization
- Manage output file operations

Key Methods:

```
def saveData(data: Dataset[ProcessedMovieRecord], outputPath: String):
Unit
```

Design Principles:

- **Immutability**: No side effects in data processing
- **Type Safety**: Strong typing for data structures

Functional Programming Implementation

1. Pure Functions

DataParsingFunctions

```
def parseMovieRecord(line: String): Option[MovieRecord] def
isValidMovieRecord(record: MovieRecord): Boolean
```

- **Purity**: No side effects, deterministic output
- **Error Handling**: Returns Option for safe parsing

DataFilteringFunctions

```
def isHighQualityMovie(record: MovieRecord): Boolean def
isFromDecade(decade: String)(record: MovieRecord): Boolean def
hasMinimumRating(minRating: Double)(record: MovieRecord): Boolean
```

- **Currying**: Partial application for flexible filtering
- **Composability**: Functions can be combined

DataTransformationFunctions

```
def enrichMovieRecord(record: MovieRecord): ProcessedMovieRecord def
categorizeRating(rating: Double): String def calculateDecade(year: Int):
String def computePopularityScore(rating: Double, votes: Long): Double
```

- **Immutability**: Creates new objects instead of modifying existing ones
- **Pattern Matching**: Used in rating categorization

2. Higher-Order Functions

Spark Transformations

```
val filteredMovies = movieRecords
.filter(DataFilteringFunctions.isHighQualityMovie(_))
.map(DataTransformationFunctions.enrichMovieRecord)
```

Custom Combinators

```
def when[A](condition: Boolean)(f: A => A): A => A def pipe[A](value:
A)(f: A => A): A def andThen[A, B, C](f: A => B)(g: B => C): A => C def
compose[A, B, C](f: B => C)(g: A => B): A => C
```

3. Tail Recursion

DataAggregationFunctions

```
def computeAverageRatingTail(movies: List[ProcessedMovieRecord], sum:
Double = 0.0, count: Int = 0): Double
```

- **Tail Recursion**: Optimized for large datasets
- **Stack Safety**: Prevents stack overflow

4. Functional Error Handling

Option Type Usage

```
def parseMovieRecord(line: String): Option[MovieRecord] = { // Returns
Some(record) on success, None on failure }
```

Either Type for Complex Error Handling

```
def retry[A](maxAttempts: Int)(f: () => A): Either[Throwable, A]
```

Data Flow

1. Input Stage

```
CSV File → DataLoader → Dataset[String]
```

2. Parsing Stage

```
Dataset[String] → parseMovieRecord → Dataset[Option[MovieRecord]] →  
filter(_.isDefined) → Dataset[MovieRecord]
```

3. Filtering Stage

```
Dataset[MovieRecord] → filter(isHighQualityMovie) → Dataset[MovieRecord]
```

4. Transformation Stage

```
Dataset[MovieRecord] → map(enrichMovieRecord) →  
Dataset[ProcessedMovieRecord]
```

5. Output Stage

```
Dataset[ProcessedMovieRecord] → DataSaver → CSV Files
```

6. Analytics Stage (Optional)

```
Dataset[ProcessedMovieRecord] → groupByKey(_.movie.genre) →  
Dataset[GenreStatistics] Dataset[ProcessedMovieRecord] →  
groupByKey(_.decade) → Dataset[(String, List[ProcessedMovieRecord])]
```

Configuration

Build Configuration (build.sbt)

```
name := "functional-data-pipeline" version := "1.0.0" scalaVersion :=
"2.12.19" // Dependencies libraryDependencies += Seq( "org.apache.spark"
%% "spark-core" % "3.5.0", "org.apache.spark" %% "spark-sql" % "3.5.0",
"org.scalatest" %% "scalatest" % "3.2.17" % Test, "org.typelevel" %%
"cats-core" % "2.10.0" ) // JVM Options run / fork := true run /
javaOptions += Seq( "--add-opens=java.base/java.nio=ALL-UNNAMED",
"--add-opens=java.base/sun.nio=ALL-UNNAMED" )
```

Runtime Configuration

PipelineConfig

```
case class PipelineConfig( minRating: Double = 7.0, minVotes: Long =
100000L, topMoviesCount: Int = 5 )
```

Spark Configuration

- **Master**: local[*] (local mode with all available cores)
- **App Name**: "Functional Data Processing Pipeline"
- **Memory**: 7.0 GiB (configurable)

Testing Strategy

Unit Testing Framework

- **ScalaTest**: Primary testing framework
- **Test Structure**: Organized by component
- **Coverage**: All pure functions tested

Test Categories

1. DataParsingFunctionsTest

```
"parseMovieRecord" should "parse valid CSV line correctly"  
"parseMovieRecord" should "return None for invalid input"  
"isValidMovieRecord" should "validate movie record fields"
```

2. DataFilteringFunctionsTest

```
"isHighQualityMovie" should "filter movies by rating and votes"  
"isFromDecade" should "filter movies by decade using currying"  
"hasMinimumRating" should "filter movies by minimum rating"
```

3. CombinatorsTest

```
"when combinator" should "apply function conditionally" "pipe combinator"  
should "chain function applications" "retry combinator" should "retry  
operations with backoff"
```

Test Execution

```
sbt test
```

Deployment

Prerequisites

- **Java**: OpenJDK 11
- **Scala**: 2.12.19
- **Sbt**: 1.9.7
- **Memory**: Minimum 4GB RAM

Build Process

```
# Compile sbt compile # Run tests sbt test # Create JAR sbt assembly
```

Execution

```
# Direct execution sbt "runMain com.functionalpipeline.Main  
data/input/sample_movies.csv data/output" # Using run script ./run.sh
```

Output Structure

```
data/output/ ■■■ _SUCCESS ■■■ part-00000-*.csv
```

Performance Considerations

Memory Management

- **Spark Memory**: 7.0 GiB allocated
- **Garbage Collection**: G1GC for better performance
- **Caching**: Strategic caching of frequently accessed data

Scalability

- **Local Mode**: Suitable for development and small datasets
- **Cluster Mode**: Can be extended for production use
- **Partitioning**: Automatic partitioning by Spark

Optimization

- **Lazy Evaluation**: Spark's lazy evaluation for efficiency
- **Predicate Pushdown**: Filter operations pushed to data source
- **Code Generation**: Spark's code generation for performance

Error Handling

Input Validation

- **CSV Parsing**: Graceful handling of malformed data
- **Type Safety**: Compile-time type checking
- **Option Types**: Safe handling of missing data

Runtime Errors

- **Resource Management**: Proper cleanup of Spark resources
- **Exception Handling**: Try-catch blocks for critical operations
- **Logging**: Comprehensive logging for debugging

Future Enhancements

Potential Improvements

1. **Streaming Processing**: Real-time data processing
2. **Machine Learning**: MLlib integration for predictive analytics
3. **Graph Processing**: GraphX for relationship analysis
4. **Cloud Deployment**: AWS/Azure integration
5. **Monitoring**: Application performance monitoring

Scalability Options

1. **Cluster Mode**: Multi-node Spark cluster
2. **Data Partitioning**: Optimized data partitioning strategies
3. **Caching**: Strategic data caching for performance
4. **Compression**: Data compression for storage efficiency

Conclusion

This Functional Data Processing Pipeline demonstrates the effective application of functional programming principles in a real-world data processing scenario. The design emphasizes immutability, pure functions, and composability while leveraging Apache Spark's distributed computing capabilities for scalable data processing.

The architecture is modular, testable, and maintainable, making it suitable for both educational purposes and production use with appropriate modifications for scale and performance requirements.

****Document Version:**** 1.0

****Last Updated:**** September 2025

****Author:**** Functional Programming Project Team